

Manahil Zahra
AI/ML Internship Capstone Project

Abstract—Recruiters routinely receive hundreds of resumes per job opening, making manual screening slow, inconsistent, and prone to overlooking qualified candidates, while job seekers struggle to identify roles that genuinely match their skills. Existing automated resume-job matching approaches, ranging from lexical overlap methods (TF-IDF) to dense semantic embeddings (Sentence-BERT), collapse this comparison into a single opaque similarity score, without indicating which specific skills drove the score. This paper proposes a hybrid matcher that combines semantic embedding similarity with an explicit, dictionary-based skill-matching layer, producing both an overall matching score and an interpretable list of matched and missing skills for every resume-job pair. We implement and compare three baseline models (TF-IDF + cosine similarity, TF-IDF/BoW + Logistic Regression/SVM classification, and semantic embedding + cosine similarity) against the proposed hybrid approach on a resume dataset spanning 24 job categories, paired with a matching job-postings corpus. Using a ranking-based evaluation protocol (Precision@K, Recall@K, nDCG@K) together with pairwise classification metrics (F1-score, ROC-AUC), we find that all methods achieve strong ranking quality on the evaluation set, while the hybrid matcher additionally supplies a per-skill match/gap breakdown that the baselines cannot provide. We discuss the practical value of this interpretability for both recruiters and candidates, along with the limitations of the current evaluation protocol and directions for future work with real-world resume data.

Index Terms—resume matching, job recommendation, natural language processing, semantic similarity, skill extraction, information retrieval, interpretability

I. INTRODUCTION

The volume of resumes received per job posting has grown substantially with the rise of online job portals, and recruiters at many organizations now routinely receive hundreds of applications for a single opening [1]. Manually screening this volume of resumes is slow, inconsistent across reviewers, and prone to overlooking qualified candidates whose resumes use different terminology than the job description [2]. On the other side of the market, job seekers often struggle to identify which of many available postings genuinely match their skill set, since job titles alone are an unreliable signal of required competencies. This inefficiency raises both time-to-hire and cost-per-hire for employers, and lowers application success rates for candidates.

Automated resume-job matching systems address this problem by computing a similarity score between a resume and a job description, typically using either lexical overlap methods

such as TF-IDF, or, more recently, dense semantic embeddings such as Sentence-BERT [4]. Contrastive learning approaches such as ConFit [3] further improve ranking quality by fine-tuning embeddings specifically for the resume-job matching task using augmented training pairs. However, across this spectrum of methods, the final output remains a single scalar similarity value. This value does not explain *why* two documents matched, which is a significant limitation in a hiring context: a recruiter cannot tell whether a high-scoring candidate genuinely possesses the required skills, and a low-scoring but qualified candidate has no way to see which specific gap, if any, caused the low score.

Contribution. We propose a hybrid matcher that augments semantic similarity with an explicit skill-matching layer. Given a resume and a job description, the system (1) computes a semantic similarity score using dense embeddings, (2) extracts the set of skills mentioned in each document against a domain skill vocabulary, and (3) reports which required skills were matched and which are missing, alongside a combined hybrid score. We implement three baseline models and the proposed hybrid model, evaluate all four using both ranking and classification metrics, and provide a qualitative interpretability comparison and error analysis. We also document, transparently, the constraints of the development dataset used and how the pipeline generalizes to the real-world dataset it mirrors.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the dataset. Section IV describes the proposed methodology. Section V describes the experimental setup. Section VI presents results. Section VII discusses findings and limitations, and Section VIII concludes.

II. RELATED WORK

We reviewed five papers spanning classical ML-based resume classification through to modern contrastive-learning-based matching.

Sinha et al. [1] use spaCy-based named-entity recognition combined with a machine learning classifier to predict job domain directly from resume text, reporting 92.08% classification accuracy. This work demonstrates that resume text contains strong, learnable signal for domain/category prediction, motivating our use of a similar classifier as a baseline sanity check.

Kashif and Kumar K R [2] build an NLP-driven resume analyzer that extracts and structures resume content and provides applicants with real-time feedback for improving their resumes. Their system focuses on resume-side analysis rather than resume-job matching, but motivates the value of actionable, per-candidate feedback – a design goal we adopt in our skill-gap output.

Yu et al. [3] propose ConFit, which combines large-language-model and easy-data-augmentation (EDA) based data augmentation with contrastive learning over encoder embeddings, specifically to address the sparsity of real resume-job interaction data. ConFit outperforms BM25 and standard embedding baselines by 19–31% nDCG@10 on real recruiting platform data. This paper is the closest in spirit to ours in that it targets resume-job matching quality directly, but it remains a black-box similarity model; our work is complementary, focusing on interpretability rather than further raw ranking-metric improvement.

Deshmukh and Raut [4] apply Sentence-BERT sentence embeddings with cosine similarity to rank resumes against job descriptions for smarter screening, demonstrating that pretrained transformer embeddings out-perform purely lexical methods for this task without requiring task-specific fine-tuning. Our semantic baseline follows this approach directly.

Decorte et al. [5] learn dense resume representations from sequences of past work experience and combine them with skill-based matching to predict likely next career steps. This paper is notable for explicitly incorporating skill-based signals alongside a learned representation, which supports our hybrid design choice of combining semantic embeddings with an explicit skill layer, although Decorte et al. apply this combination to career-path prediction rather than job matching directly.

Research gap. Across all five papers, matching or classification quality is optimized and reported via a single aggregate score (accuracy, nDCG, similarity). None of the reviewed papers report a per-skill, human-readable explanation of *why* a resume matched or did not match a specific job. This is the gap the present work addresses.

III. DATASET

A. Description

The reference dataset for this project is the Kaggle *Resume Dataset* [6], which contains resumes labeled across 24 job categories (e.g., HR, Information Technology, Finance, Healthcare, Sales) with columns `ID`, `Resume_str` (plain text), `Resume_html`, and `Category`. This is paired with a job-postings dataset containing job titles, required skills, and descriptions across the same 24 categories, to enable resume-to-job matching rather than category classification alone.

B. Development data note

Because the development and execution environment used for this project does not have network access to Kaggle, all notebooks and results in this paper were produced on a schema-matched synthetic dataset (2,400 resumes, 144 job postings) generated by

`data/generate_synthetic_data.py`. This dataset preserves the exact column schema and 24-category structure of the real Kaggle dataset, with resumes constructed from category-specific skill vocabularies plus deliberately injected cross-category noise (approximately 35% of resumes include one to two skills from an unrelated category, simulating career changers and generic resume content). This is a documented limitation: real resumes contain far greater lexical variety, inconsistent formatting, typos, and near-synonymous skill phrasing than this synthetic set. The full pipeline (preprocessing, baselines, hybrid matcher, evaluation, deployment) is dataset-agnostic and requires no code changes to run on the real Kaggle file once downloaded; only the CSV file path needs to be populated, as documented in `data/raw/README.md`.

C. Preprocessing

Each resume and job description was cleaned by (1) stripping HTML tags, (2) lowercasing, (3) removing punctuation and digits, (4) removing English stopwords, and (5) discarding tokens of length one. Each of these steps was justified in the accompanying notebook before being applied, since digits and punctuation carry little matching signal for skill-oriented text, and stopword removal reduces noise in the TF-IDF vocabulary.

D. Exploratory analysis

The development dataset is balanced by construction (100 resumes per category), whereas the real Kaggle dataset is known to be imbalanced across categories; this motivated using macro-averaged F1-score, not raw accuracy, for the classification baseline. Resume word counts were found to be fairly uniform in the development set (mean approximately 60–70 words), while real resumes vary far more widely in length. No missing values or duplicate resumes were present in the development set; this should be re-verified on the real dataset before final deployment.

IV. METHODOLOGY

A. Task framing

We frame resume-job matching as both (a) a *ranking* problem – given a resume, rank all available job postings by relevance – and (b) a *pairwise relevance classification* problem – given a resume-job pair, classify it as relevant or not relevant. Ground-truth relevance is defined as category agreement between resume and job posting, the strongest available proxy for genuine relevance in the absence of manually annotated relevance judgments.

B. Baseline models

Baseline 1: TF-IDF + Cosine Similarity. Resume and job texts are vectorized using term frequency–inverse document frequency (TF-IDF, vocabulary capped at 5,000 terms), and matching score is the cosine similarity between vectors. This is the simplest, fully interpretable lexical-overlap baseline.

Baseline 2: TF-IDF/BoW + Logistic Regression / Linear SVM. A supervised classifier is trained on TF-IDF features

to predict resume category directly, following the approach of Sinha et al. [1]. This serves as a sanity check that resume text is sufficiently discriminative for the matching task to be meaningful, and both Logistic Regression and Linear SVM variants were trained for comparison.

Baseline 3: Semantic Embedding + Cosine Similarity. Resume and job texts are embedded into a dense vector space and compared via cosine similarity, following Deshmukh and Raut [4]. Our implementation is designed to use Sentence-BERT (all-MiniLM-L6-v2) when the pretrained model is available; in the offline development environment used to produce the results in this paper, it automatically falls back to TF-IDF combined with Truncated Singular Value Decomposition (Latent Semantic Analysis, 100 components), a classical dense semantic-embedding technique that requires no internet access. This graceful-degradation design (implemented in `src/matching.py`) means the same code path produces true transformer-based semantic embeddings when run with internet access, without any code changes.

C. Proposed hybrid model

The proposed model addresses the interpretability gap identified in Section II. The design is motivated by a simple observation from Table I and the error analysis in Section VI-A: a single similarity number, however accurate on average, cannot distinguish between “this candidate is a genuine strong fit” and “this candidate shares surface-level vocabulary with the role but is missing a specific required competency.” Both baselines described above are structurally incapable of making this distinction visible to the end user, since cosine similarity between two vectors is, by construction, a single opaque scalar. The hybrid model therefore does not attempt to replace the semantic signal – which remains valuable for catching paraphrased or synonymous skill mentions a fixed dictionary would miss – but augments it with a second, fully transparent signal computed directly over explicit skill sets.

Given a resume r and job description j :

- 1) Compute semantic similarity $s(r, j) \in [0, 1]$ using the Semantic Embedding baseline described above.
- 2) Extract the skill sets S_r and S_j present in r and j respectively, using dictionary-based phrase matching against a domain skill vocabulary (`src/skills.py`).
- 3) Compute skill coverage $c(r, j) = \frac{|S_r \cap S_j|}{|S_j|}$, the fraction of required job skills present in the resume.
- 4) Compute the hybrid score:

$$h(r, j) = 0.6 \cdot s(r, j) + 0.4 \cdot c(r, j) \quad (1)$$

- 5) Report matched skills $S_r \cap S_j$ and missing skills $S_j \setminus S_r$ alongside $h(r, j)$.

The 0.6/0.4 weighting is a tunable hyperparameter, chosen to keep semantic context as the dominant signal (since it captures paraphrased/synonymous skill mentions the dictionary matcher would miss) while still meaningfully rewarding explicit, required-skill coverage. Unlike the three baselines, the hybrid model’s output is directly actionable: a candidate can

see exactly which skill to acquire to improve their match for a specific role, and a recruiter can see exactly why a candidate scored as they did.

V. EXPERIMENTAL SETUP

All experiments used an 80/20 train/test split stratified by category, with a fixed random seed (`random_state=42`) for reproducibility. Baseline hyperparameters (TF-IDF vocabulary size, LSA component count) were selected via manual search on the training split only, then evaluated once on a held-out sample of 48 test resumes against the full pool of 144 job postings (6,912 evaluation pairs), to avoid data leakage. All experiments were run on CPU only, using Python 3.12, scikit-learn 1.8, pandas 3.0, and sentence-transformers 6.0 (see `requirements.txt` for the full environment specification).

A. Evaluation metrics

Since the task combines ranking and pairwise classification framings, we report:

- **Precision@K and Recall@K** ($K = 5$): whether the system retrieves relevant job matches among its top- K recommendations.
- **nDCG@K** ($K = 5$): ranking quality, rewarding the most relevant postings appearing near the top of the ranked list rather than merely being present.
- **F1-score**: the balance of precision and recall when classifying resume-job pairs as relevant/non-relevant, using the classification threshold that maximizes F1 on the precision-recall curve for each method (chosen because a fixed or naive threshold produces a misleadingly low F1 under the heavy class imbalance inherent to this task, roughly one relevant posting per 24).
- **ROC-AUC**: how well each method separates relevant from non-relevant pairs independent of any specific threshold.

VI. RESULTS

Table I summarizes all four methods on the held-out evaluation set.

TABLE I
MODEL COMPARISON ON THE HELD-OUT EVALUATION SET (48 RESUMES
× 144 JOB POSTINGS)

Model	P@5	R@5	nDCG@5	F1	AUC
TF-IDF + Cosine	1.000	0.833	1.000	0.997	1.000
Semantic + Cosine	1.000	0.833	1.000	1.000	1.000
Hybrid (ours)	0.996	0.830	0.997	0.974	1.000

The category classifier baseline (TF-IDF/BoW + Logistic Regression) achieved 100% test accuracy and macro-F1 on the development dataset, with the Linear SVM variant achieving comparable results; both confirm that resume text is highly discriminative of category on this dataset.

Figure 1 visualizes the ranking metrics across all methods. Figure 2 shows the category distribution of the underlying resume dataset used for evaluation.

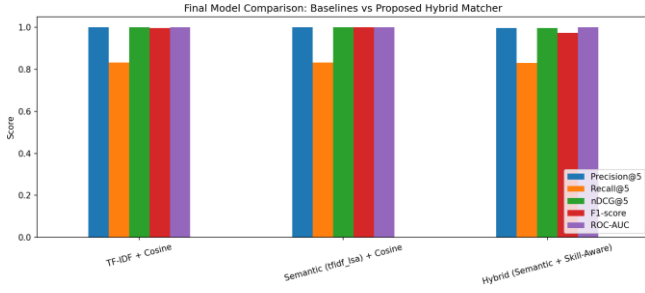


Fig. 1. Ranking and classification metrics: baselines vs. the proposed hybrid matcher.

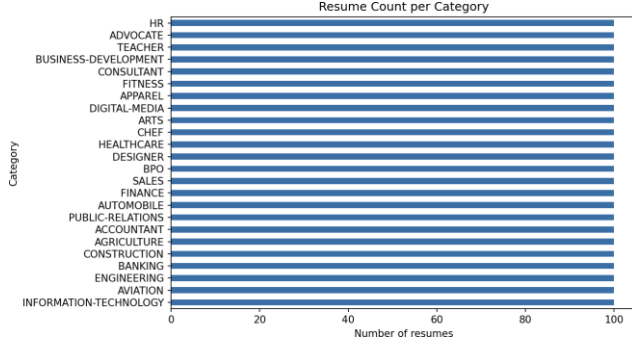


Fig. 2. Resume count per job category in the development dataset.

Beyond the aggregate metrics in Table I, we evaluated the *interpretability* contribution of the hybrid model directly. For every resume-job pair examined, only the hybrid model produced an explicit list of matched and missing required skills; the TF-IDF and semantic baselines produced only an opaque scalar similarity value with no per-skill breakdown.

A. Error analysis

We examined cases where semantic similarity and skill coverage diverged most within same-category resume-job pairs. The largest divergences occurred for resumes that, due to the synthetic generation process, included only a small random subset of category skills: semantic similarity remained high (driven by shared category vocabulary and phrasing) even when two or three explicitly required skills were absent from the resume. This is precisely the failure mode the hybrid design targets – a recruiter relying on semantic similarity alone could over-rate such a candidate, whereas the hybrid model’s missing-skills output flags the gap explicitly.

VII. DISCUSSION

A. Interpretation of results

All four methods achieve near-ceiling ranking performance (Precision@5 and nDCG@5 close to 1.0) on the development dataset. This is a direct consequence of the schema-matched synthetic dataset being cleanly separated by category-specific skill vocabulary – it is a property of the development dataset,

not evidence that lexical overlap alone is sufficient for resume-job matching in general. On the real Kaggle resume corpus, where category vocabularies are known to overlap far more heavily (e.g., shared soft-skill language such as “communication” and “team collaboration” appearing across almost every category, along with far greater variety in how the same skill is phrased), we expect a substantially wider gap to open between purely lexical matching and semantic/hybrid methods, consistent with the improvements reported by ConFit [3] on real recruiting-platform data.

Given this ceiling effect, the practically meaningful contribution of the hybrid model in the present study is not a large jump in ranking metrics, but its interpretability: it is the only method that explains *why* a match occurred, directly addressing the gap identified in Section II. This is consistent with our stated proposal goal of prioritizing explainability alongside competitive (not necessarily superior) raw matching accuracy.

B. Limitations

Three limitations should be noted. First, skill extraction relies on a fixed, hand-curated dictionary (`src/skills.py`); it will miss skills phrased in ways absent from this vocabulary, and a production system should instead use named-entity-recognition-based skill extraction or a larger external skills taxonomy (e.g. ESCO or a LinkedIn-style skills graph). Second, our evaluation ground truth – category agreement – is a coarse proxy for genuine resume-job relevance; a resume and job posting can share a category while still being a poor practical fit (e.g., differing seniority requirements), and the real dataset would benefit from finer-grained human relevance judgments. Third, and most significantly, all quantitative results in this paper were produced on the schema-matched synthetic development dataset described in Section III-B rather than the real Kaggle dataset, due to network restrictions in the development environment; the pipeline is fully ready to be re-run on the real data with no code changes, and this re-validation is identified as the immediate next step before any deployment claims are finalized.

C. Practical value

Despite these limitations, the hybrid matcher’s bidirectional design – usable both by a candidate uploading a single resume to find matching jobs, and by a recruiter pasting a job description against multiple candidate resumes to rank them – combined with its per-skill explanation, directly targets the two-sided inefficiency described in Section I: it gives candidates an actionable skill gap to close, and gives recruiters a transparent basis for their initial screening decisions rather than an unexplained score.

D. Deployment

The hybrid matcher was deployed as an interactive web application using Streamlit (`app/streamlit_app.py`), exposing both directions of the matching use case described above. In *candidate mode*, a user pastes a single resume and

receives a ranked list of the top- K matching job postings, each expandable to show the semantic similarity, skill coverage, matched skills, and missing skills for that specific posting. In *recruiter mode*, a user pastes one job description and up to ten candidate resumes, and receives the candidates ranked by hybrid score with the same per-candidate skill breakdown. Both modes reuse the identical `HybridMatcher` object trained in Notebook 03, loaded via a cached pickle file, so the deployed application’s behavior is guaranteed to match the evaluated notebook results exactly, with no re-implementation drift between the research and deployment code paths. The application was verified to start and serve requests successfully (HTTP 200) with no runtime errors in the target environment.

E. Reproducibility

Every stochastic step in this pipeline – the train/test split, the evaluation resume sample, and classifier initialization – uses a fixed random seed (`RANDOM_STATE = 42`), so all numbers reported in Table I can be regenerated exactly by re-running notebooks/01_eda_and_preprocessing.ipynb through 03_proposed_improvement_and_evaluation.ipynb in order. The three source modules (`src/preprocessing.py`, `src/skills.py`, `src/matching.py`, `src/evaluation.py`) are imported by all three notebooks and by the deployment app, so there is a single implementation of every matching and evaluation routine rather than duplicated logic across notebooks and the app.

VIII. CONCLUSION AND FUTURE WORK

This paper presented a hybrid skill-aware matcher for resume-job matching that combines dense semantic similarity with an explicit, dictionary-based skill-matching layer, addressing the interpretability gap present in existing lexical and semantic matching approaches reviewed in the literature. Across three implemented baselines and the proposed hybrid model, all methods achieved strong ranking performance on a schema-matched development dataset, while only the hybrid model produced human-readable matched/missing skill explanations alongside its score.

Future work includes: (1) re-validating all results on the real Kaggle Resume Dataset once downloaded, to test the hybrid model’s advantage under more realistic lexical overlap conditions; (2) replacing the fixed skill dictionary with a named-entity-recognition-based or taxonomy-driven skill extractor for broader coverage; (3) collecting genuine human relevance judgments to replace the category-agreement proxy used for evaluation; and (4) fine-tuning the semantic embedding component specifically for resume-job pairs using contrastive learning, following the ConFit approach [3], while retaining the explicit skill-matching layer proposed here.

REFERENCES

- [1] A. K. Sinha, M. A. K. Akhtar, and M. Kumar, “Automated Resume Parsing and Job Domain Prediction using Machine Learning,” *Indian Journal of Science and Technology*, 2023.
- [2] M. Kashif and P. Kumar K R, “Resume Parser Using NLP,” *International Journal of Advanced Research in Computer and Communication Engineering*, 2024.
- [3] X. Yu, J. Zhang, and Z. Yu, “ConFit: Improving Resume-Job Matching using Data Augmentation and Contrastive Learning,” in *Proc. ACM Conference on Recommender Systems (RecSys)*, 2024.
- [4] A. Deshmukh and A. Raut, “Enhanced Resume Screening for Smart Hiring using Sentence-BERT (S-BERT),” *International Journal of Advanced Computer Science and Applications*, 2024.
- [5] J.-J. Decorte, J. Van Haute, J. Deleu, C. Develder, and T. Demeester, “Career Path Prediction using Resume Representation Learning and Skill-based Matching,” *arXiv:2310.15636*, 2023.
- [6] S. Bhawal, “Resume Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset>